DroneArjuna GCS

Connecting Your Drone Firmware to DroneArjuna

Clear step-by-step instructions. DroneArjuna is a standard MAVLink ground station, so the firmware needs no DroneArjuna-specific code - it only has to speak MAVLink.

Part A	Firmware side: enable MAVLink and meet the minimum message requirements
Part B	Link the drone to the PC (USB / SiK radio, or UDP)
Part C	DroneArjuna side: register the drone and connect
Part D	Verify, bench test, first flight
Appendix	Optional signing, troubleshooting, checklist

Part A - Firmware side

Step 1. Enable MAVLink output on one port

Stock ArduPilot

Parameter	Value
SERIAL1_PROTOCOL (or the port you use)	2 (MAVLink 2)
SERIAL1_BAUD	57 for a SiK radio (57600); 115 for 115200. USB is fixed at 115200.
SYSID_THISMAV	A unique number, 1-250 (one per vehicle)

PX4: enable a MAVLink instance on the chosen port and set `MAV_SYS_ID` to a unique number.

NOTE: These parameter names come from general ArduPilot/PX4 knowledge, not from the DroneArjuna repository. Confirm them for your firmware version.

Step 2. Custom firmware: minimum MAVLink requirements

If your firmware is not ArduPilot or PX4, it must implement the following for DroneArjuna to work properly:

Requirement	Why DroneArjuna needs it
Send HEARTBEAT at about 1 Hz	Connection fails if no first heartbeat arrives within 10 s (45 s on HF)
Set autopilot and type correctly in HEARTBEAT	Mode names are decoded from these. Only ArduCopter, ArduPlane and PX4 tables exist; other modes show as MODE_<n>
Send GLOBAL_POSITION_INT, ATTITUDE, VFR_HUD, SYS_STATUS, GPS_RAW_INT, BATTERY_STATUS, RC_CHANNELS	These are the telemetry messages the backend reads. Missing ones leave that UI value at its default
Honour MAV_CMD_SET_MESSAGE_INTERVAL and REQUEST_DATA_STREAM	The backend requests GPS_RAW_INT 2 Hz, RC_CHANNELS 2 Hz, GLOBAL_POSITION_INT 4 Hz, ATTITUDE 10 Hz, VFR_HUD 4 Hz, SYS_STATUS 2 Hz
Reply to COMMAND_LONG with COMMAND_ACK (arm/disarm, takeoff, flight termination, set home)	Backend waits up to 5 s for the ACK, 2 retries; otherwise the command times out
Accept SET_MODE, MISSION_ITEM_INT (guided goto), SET_POSITION_TARGET_LOCAL_NED	Carry set_mode / rtl / land, goto and velocity commands
Accept MAV_CMD_DO_SET_HOME	Home is pushed on connect and every 5 s from vessel position (ship RTL)
MAVLink v2 (with signing if enabled)	Signing is applied by the backend when MAVLINK_SIGNING_KEY is set

Messages DroneArjuna consumes: HEARTBEAT, GLOBAL_POSITION_INT, ATTITUDE, VFR_HUD, SYS_STATUS, GPS_RAW_INT, RC_CHANNELS, BATTERY_STATUS, COMMAND_ACK.

Part B - Link the drone to the PC

Option 1: USB cable or SiK telemetry radio (Windows)

- Plug in the USB cable or ground radio. Close Mission Planner, QGroundControl or any program holding the COM port.
- In the DroneArjuna folder run: `.\start.ps1` (auto-detects a USB serial port at 115200). For a SiK radio on COM4: `.\start.ps1 -ComPort COM4 -Baud 57600`.
- Open `http://localhost:5761/ports` - it must show `"connected": true` and your COM port.
- Why: Docker on Windows cannot open COM ports, so `com_bridge.py` forwards the port to TCP 5762 for the backend.

Option 2: UDP (Wi-Fi, LTE, companion computer)

- Make the vehicle or companion computer send MAVLink to `<PC_IP>:14550` over UDP. Example: `mavproxy.py --master=/dev/ttyAMA0,921600 --out=udp:<PC_IP>:14550`
- In `.env` set `RADIO_BIND_ADDR=<PC LAN IP>`. The default 127.0.0.1 blocks remote drones.
- Allow UDP 14550 through the Windows firewall, then recreate the Docker stack.
- The backend listens, so the vehicle must transmit first.

Part C - DroneArjuna side

Step 1. Register the drone

- Log in at `http://localhost:3000` as FLIGHT_CONTROLLER or higher.
- Go to **Settings -> Drone Instance Manager** and add the drone: call sign, type, serial number, and **MAVLink ID = your SYSID_THISMAV**.
- Keep it active and not in 'maintenance', otherwise auto-connect skips it.

Step 2. Connect

- **Automatic:** the backend detects the bridge and connects in about 7 s.
- **Fleet UI:** Fleet -> Connect -> choose a preset: **Windows COM Bridge TCP 5762** for USB/SiK, or **SITL UDP 14550** for UDP (the name says SITL but it is plain UDP port 14550).
- **API:** see below.

```
POST /api/drone-control/connect                  (USB / SiK via bridge)
{ "drone_instance_id": 3, "transport": "tcp",
  "host": "host.docker.internal", "port": 5762 }

POST /api/drone-control/connect                  (UDP)
{ "drone_instance_id": 3, "transport": "udp", "port": 14550 }

POST /api/drone-control/autoconnect              (probe all candidates)
{ "drone_instance_id": 3 }
```

Success: `201 Connected`. Failure: `503` heartbeat timed out.

Part D - Verify and fly

- Fleet shows the drone connected; position, attitude, battery and GPS update. `GET /api/drone-control/status` lists it.
- Bench test with propellers removed: arm/disarm, mode change, RTL.
- Arm is refused below 20% battery, or without a 3D GPS fix and at least 6 satellites.
- Fly in an open area with RC override ready. Configure the autopilot's own failsafes - DroneArjuna drops a silent link after 10 s, but the vehicle must protect itself.

SAFETY: Never rely on the GCS link as the only failsafe.

Appendix A - Optional MAVLink 2 signing

- Generate a key: `python -c "import secrets;print(secrets.token_hex(32))"`; put it in .env as `MAVLINK_SIGNING_KEY` (64 hex chars) and restart the backend.
- Load the **same key into the autopilot**. If keys differ, there is no data (heartbeat timeout). Only RADIO_STATUS is accepted unsigned.

- Enable signing last, after the link works unsigned.

Appendix B - Troubleshooting

Symptom	Fix
503 heartbeat timeout	Check the firmware sends MAVLink on that port/baud; vehicle sends to PC:14550; RADIO_BIND_ADDR; firewall
Bridge connected:false	Free the COM port (close other GCS); check bridge.err.log; give an explicit COM port
Bridge: another MAVLink client connected	Only one client allowed on TCP 5762; disconnect the other
Heartbeat OK but no GPS / RSSI values	Firmware ignores stream requests or lacks the message; check message intervals / SRn_* params
Connects then drops after about 10 s	Baud mismatch or one-way link; vehicle must receive the GCS heartbeat
No data after setting signing key	Load same key on autopilot, or clear MAVLINK_SIGNING_KEY
Arm denied	Wait for 3D GPS fix (6+ sats); charge battery above 20%
Drone does not auto-connect	Create/activate the drone instance; not in maintenance
409 already connected	POST /api/drone-control/disconnect/{id}, then reconnect

Appendix C - Checklist

- [ ] MAVLink enabled on the firmware port; unique system id; GPS and battery healthy
- [ ] Custom firmware: heartbeat, telemetry messages, COMMAND_ACK, stream-interval support implemented
- [ ] USB/SiK: .\start.ps1 run, bridge shows connected:true
- [ ] UDP: vehicle sends to PC:14550, RADIO_BIND_ADDR set, firewall open
- [ ] Drone instance created and active, MAVLink ID matches the firmware system id
- [ ] Connected; telemetry updating; bench test done with props off